Analysis of the Implementation Process Report (Haoran Ma)

Program 1: Multiset Implementation

The Multiset class is implemented as a binary search tree (BST)-based data structure to store multiple occurrences of elements efficiently. It supports various operations such as insertion, deletion, and querying, ensuring logarithmic time complexity in most cases.

Key Features

- Efficient Insertion and Deletion: Maintains BST properties for optimized search and modification.
- Dynamic Count Management: Tracks occurrences of each element efficiently.
- Range and Query Operations: Supports Contains, Count, Min, Max, Floor, and Ceil functions for flexibility.

Complexity Analysis

Multiset Operations Complexity

- Insertion (Insert): O(log N) - Maintains BST properties, ensuring logarithmic complexity in a balanced tree.
- Deletion (Remove): O(log N) - Removes elements while preserving tree balance.
- Search (Contains): O(log N) - Binary search ensures quick lookups.
- Count (Count): O(log N) - Efficiently retrieves occurrences.
- Min/Max (Min, Max): O(log N) - Traverses leftmost or rightmost nodes.
- Range Queries (Floor, Ceil): O(log N) - Uses binary search for fast range operations.

While these operations are O(log N) on average, an unbalanced BST can degrade to O(N). To maintain performance, self-balancing trees like AVL or Red-Black trees can be used.

(next page program two)

Program 2: Prime Factorization using Custom Multiset

This program performs prime factorization and stores the prime factors in a custom multiset implemented using a sorted std::vector. The MyMultiset class ensures efficient storage and retrieval.

Key Features

- Sorted Factor Storage: Uses binary search for optimized insertion and retrieval.
- Efficient Prime Factorization: Implements trial division for factorization.
- Comprehensive Query Functions: Supports min/max retrieval and range searches.

Complexity Analysis

Prime Factorization and Query Complexity

- Insertion (insert): O(log N) - Uses binary search (std::lower_bound) to locate insertion points.
- Factorization (factorize): O(sqrt(N)) - Trial division ensures efficient prime factorization.
- Min/Max (min, max): O(1) - Retrieves smallest and largest elements instantly.
- Search (find): O(log N) - Binary search enables fast lookups.
- Range Queries (lower_bound_gt, upper_bound_lt): O(log N) - Uses binary search for efficient nearest-element retrieval.

While O(sqrt(N)) is optimal for moderate numbers, advanced methods like Pollard's rho algorithm or Sieve of Eratosthenes could improve performance for larger numbers.

Conclusion

Both implementations demonstrate efficient data structures for their respective tasks:

1. The BST-based Multiset provides logarithmic complexity for most operations, with balancing strategies improving worst-case performance.
2. The vector-based MyMultiset offers sorted storage with logarithmic query time, making it suitable for small to medium-sized factorization tasks.

3. Prime factorization is implemented optimally with O(sqrt(N)) complexity, with potential optimizations for larger numbers.
4. Systematic testing ensures correctness and robustness of both implementations.